

Trabalho Prático da Disciplina de Programação Web

Desenvolvimento de uma Aplicação Web Baseada em Microserviços com Spring Boot e React

1. Objetivo

O objetivo deste trabalho é desenvolver uma aplicação web completa utilizando tecnologias modernas para desenvolvimento de sistemas distribuídos.

Ao final do projeto, espera-se que os estudantes sejam capazes de:

- desenvolver APIs REST utilizando Spring Boot;
 - aplicar o padrão de arquitetura em microserviços;
 - implementar comunicação síncrona entre serviços utilizando OpenFeign;
 - implementar comunicação assíncrona utilizando RabbitMQ;
 - desenvolver uma aplicação frontend utilizando ReactJS;
 - integrar frontend e backend;
 - aplicar boas práticas de organização e arquitetura de software.
-

2. Formação das Equipes

O trabalho deverá ser desenvolvido em equipes de **até quatro estudantes**.

Todos os integrantes deverão conhecer toda a arquitetura do sistema, independentemente da divisão das tarefas durante o desenvolvimento.

3. Tema do Projeto

Cada equipe deverá desenvolver um sistema web de média complexidade.

O tema será definido pela equipe e deverá ser aprovado pelo professor.

Exemplos de temas:

- Sistema de Biblioteca

- Sistema de Clínica Médica
- Sistema de Hotel
- Sistema de Academia
- Sistema de Restaurante
- Sistema de Eventos
- Sistema de Locação de Veículos
- Sistema de Pet Shop
- Sistema de Gerenciamento Escolar
- Sistema de Controle de Estoque

O sistema escolhido deverá possuir regras de negócio suficientes para justificar a utilização de múltiplos microserviços.

4. Requisitos Técnicos Obrigatórios

Backend

O backend deverá ser desenvolvido utilizando:

- Java 21
 - Spring Boot
 - Spring Data JPA
 - Spring Web
 - Bean Validation
 - PostgreSQL
 - Spring Security com JWT
 - OpenFeign
 - RabbitMQ
 - Swagger/OpenAPI
-

Frontend

O frontend deverá ser desenvolvido utilizando:

- ReactJS
 - React Router
 - Axios
 - React Hooks
 - Material UI ou Bootstrap
-

5. Arquitetura

A aplicação deverá utilizar arquitetura baseada em microserviços.

Cada microserviço deverá possuir:

- responsabilidade única;
- banco de dados próprio (ou schema independente);
- API REST própria.

A solução deverá possuir, no mínimo, os seguintes componentes:

- Frontend React
 - Dois ou mais microserviços Spring Boot
 - RabbitMQ
 - Banco de dados PostgreSQL
-

6. Comunicação entre Microserviços

Comunicação Síncrona

Deverá existir comunicação REST entre microserviços utilizando **OpenFeign**.

Exemplos:

- consulta de informações;
 - validação de dados;
 - atualização de recursos.
-

Comunicação Assíncrona

Deverá existir comunicação utilizando **RabbitMQ**.

Pelo menos um fluxo do sistema deverá publicar mensagens em uma fila e outro microserviço deverá consumi-las.

Exemplos:

- envio de notificações;
- geração de logs;
- processamento de eventos;
- atualização de relatórios.

7. Funcionalidades Mínimas

O sistema deverá possuir:

- pelo menos três entidades principais;
 - operações completas de CRUD;
 - validação dos dados;
 - tratamento global de exceções;
 - documentação Swagger;
 - persistência em PostgreSQL.
-

8. Frontend

O frontend deverá consumir exclusivamente as APIs REST desenvolvidas pela equipe.

A aplicação deverá possuir interface gráfica para utilização das principais funcionalidades do sistema.

No mínimo deverão existir:

- tela inicial;
 - listagem de dados;
 - cadastro;
 - edição;
 - exclusão;
 - consultas.
-

9. Organização do Projeto

O backend deverá utilizar arquitetura em camadas.

Espera-se a utilização de:

- Controllers
- Services
- Repositories
- DTOs
- Mapeamento entre entidades e DTOs

- Tratamento global de exceções
 - Validação utilizando Bean Validation
-

10. Entrega

Cada equipe deverá entregar:

- código-fonte do backend;
 - código-fonte do frontend;
 - arquivo docker-compose para execução da infraestrutura necessária;
 - documentação da API (Swagger);
 - coleção Postman ou Insomnia;
 - README contendo:
 - descrição do projeto;
 - arquitetura utilizada;
 - instruções de execução;
 - tecnologias empregadas;
 - divisão das responsabilidades da equipe.
-

11. Apresentação

Na data definida pelo professor, cada equipe deverá apresentar o funcionamento geral da aplicação.

A apresentação deverá incluir:

- arquitetura do sistema;
 - funcionamento dos microserviços;
 - comunicação síncrona;
 - comunicação assíncrona;
 - autenticação;
 - integração entre frontend e backend.
-

12. Avaliação

A avaliação será **individual**, embora o desenvolvimento seja realizado em equipe.

O projeto entregue servirá apenas como base para a avaliação prática.

A nota **não será atribuída pela qualidade do código entregue**, mas pela capacidade de cada estudante compreender, modificar e evoluir o sistema durante uma arguição presencial.

Após a apresentação da equipe, cada estudante será avaliado individualmente.

O professor proporá uma modificação no sistema, que deverá ser implementada pelo estudante durante a avaliação.

Cada modificação será diferente, porém possuirá nível de dificuldade equivalente.

Durante a avaliação, o estudante deverá:

- compreender a solicitação apresentada;
- localizar os componentes envolvidos;
- implementar a modificação;
- executar o sistema demonstrando o funcionamento da solução;
- explicar as alterações realizadas;
- responder às perguntas formuladas pelo professor.

As modificações poderão envolver qualquer parte da aplicação, incluindo:

- backend;
- frontend;
- banco de dados;
- APIs REST;
- regras de negócio;
- autenticação;
- comunicação entre microserviços;
- mensageria com RabbitMQ.

O estudante deverá realizar a atividade **individualmente**, sem auxílio dos demais integrantes da equipe.

O professor poderá formular perguntas sobre qualquer parte da arquitetura do sistema.

13. Observações

- O uso de ferramentas de Inteligência Artificial é permitido durante o desenvolvimento do projeto.
- Durante a avaliação presencial, cada estudante deverá demonstrar domínio técnico sobre a solução implementada.
- Não será permitida colaboração entre os integrantes da equipe durante a arguição individual.
- O professor poderá solicitar alterações em qualquer componente da aplicação, incluindo frontend, backend, banco de dados, comunicação entre microserviços e mensageria.

- Espera-se que o projeto apresente uma arquitetura organizada, modular e de fácil manutenção, permitindo sua evolução por qualquer integrante da equipe.

Essa metodologia de avaliação busca aproximar o processo de ensino da realidade profissional, na qual desenvolvedores frequentemente precisam compreender sistemas existentes, implementar novas funcionalidades, corrigir defeitos e justificar tecnicamente as soluções adotadas.